Battle Bots Arena: Evoluzione di Strategie di Combattimento tramite Algoritmi Genetici

- Introduzione
- Tecnologie utilizzate
- Architettura del sistema
  - Struttura generale
  - Classi principali
- Implementazione dell'Algoritmo Genetico
  - **Configurazione DEAP**
  - Parametri Evolutivi
  - Flusso Evolutivo
- Codifica del Genoma
  - Tabella dei geni
- Funzione di Fitness
  - **Formula Completa**
  - **Componenti della Fitness**
  - Bilanciamento
- Operatori Genetici
  - La Selezione: Il Metodo del Torneo
- Sistema di Simulazione
- **Risultati e Osservazioni**
- Conclusioni e Sviluppi Futuri
  - Conclusioni e Prospettive Future
  - Analisi Critica
  - Sviluppi Futuri

Introduzione

Battle Bots Arena è un simulatore di combattimento robotico basato su algoritmi genetici. Il progetto vede due popolazioni di agenti (Team Rosso e Blu) evolversi in un'arena virtuale: attraverso i meccanismi di selezione, crossover e mutazione, i robot perfezionano autonomamente le proprie strategie di gioco per prevalere sugli avversari

Tecnologie utilizzate

Per lo sviluppo del progetto **Battle Bots Arena** sono state impiegate le seguenti librerie Python:

- **DEAP (Distributed Evolutionary Algorithms in Python):** Un framework flessibile utilizzato per l'implementazione e la gestione del ciclo di vita degli algoritmi genetici. Consente di definire agevolmente le strutture degli individui, le funzioni di fitness e gli operatori di selezione, crossover e mutazione.

- **Pygame:** Libreria cross-platform dedicata alla creazione di applicazioni multimediali. Nel progetto è stata sfruttata per il rendering grafico 2D dell'arena e per la gestione degli eventi in tempo reale, permettendo di visualizzare fluidamente le dinamiche di combattimento tra i robot.
- **NumPy:** Strumento fondamentale per il calcolo scientifico, utilizzato per gestire le operazioni vettoriali e le matrici di dati.

Architettura del sistema

Struttura generale

Il sistema è organizzato in tre componenti principali:

```
graph TD
    A[Battle Arena - Coordinatore] --> B[Bot]
    A --> C[Bullet]

    subgraph Coordinatore
    A1[Gestione popolazioni]
    A2[Evoluzione con DEAP]
    A3[Orchestrazione battaglie]
    end

    subgraph Bot
    B1[Genoma]
    B2[Fisica]
    B3[IA]
    end

    subgraph Bullet
    C1[Fisica]
    end
```

Classi principali

La classe **BattleArena** è il coordinatore, è la classe che:

- Gestisce il ciclo evolutivo completo;
- Coordina le battaglie tra individui;
- Implementa la logica DEAP;
- Gestisce il rendering e l’interfaccia utente.

La classe **Bot :**

- Rappresenta un singolo individuo della popolazione;
- Contiene il genoma (12 geni) ;

- Implementa la logica di movimento e combattimento;
- Traccia metriche per il calcolo del fitness.

La classe **Bullet:**

- Gestisce i proiettili sparati dai bot ;
- Implementa fisica balistica semplice;
- Controllo collisioni.

Implementazione dell'Algoritmo Genetico

Configurazione DEAP

creator.create("FitnessMax", base.Fitness, weights = (1.0,))

creator.create("Individual", list, fitness=creator.FitnessMax)

L'algoritmo è configurato per **massimizzare** la fitness (weights = (1.0,)), indicando che individui con fitness maggiore sono preferibili.

Parametri Evolutivi

Parametro	Valore	Motivazione
Dimensione popolazione	20	Bilanciamento tra diversità e velocità
Numero generazioni	100	Sufficiente per convergenza
Probabilità crossover	0.7	Standard in letteratura GA
Probabilità mutazione	0.3	Permette esplorazione continua
Tournament size	3	Pressione selettiva moderata
Sigma mutazione	0.2	Mutazioni non troppo drastiche

Flusso Evolutivo

- **Inizializzazione:** Creiamo 20 robot "rossi" e 20 "blu" con geni casuali.
- **Valutazione:** Ogni robot combatte **10 volte** (5 avversari diversi in posizioni invertite) per testare davvero quanto è forte.
- **Selezione:** Usiamo il sistema del *Torneo*: si scelgono 3 robot a caso e il migliore vince il diritto di fare figli.
- **Riproduzione:** Due robot vincitori mescolano i loro geni (*Two-point crossover*) per creare un nuovo robot.
- **Mutazione:** Alcuni geni vengono cambiati casualmente per evitare che i robot diventino tutti uguali.
- **Elitismo:** Il "campione assoluto" della generazione precedente viene sempre salvato nella nuova, così non perdiamo mai i progressi migliori.

Codifica del Genoma

Ogni bot è controllato da un **genoma di 12 geni**, ciascuno codificato come un numero reale nell'intervallo [0, 1]:

Tabella dei geni

Gene	Nome	Range effettivo	Descrizione
0	speed	1.0 - 6.0	Velocità di movimento del bot
1	aggression	0.0 - 1.0	Tendenza ad attaccare
2	shoot_freq	0.0 - 0.1	Frequenza di sparo
3	aim_skill	0.0 - 1.0	Precisione della mira
4	dodge_skill	0.0 - 1.0	Capacità di evasione
5	circle_tendency	-1.0 - 1.0	Movimento circolare (negativo=orario, positivo=antiorario)
6	keep_distance	100 - 400	Distanza preferita dal nemico (pixel)
7	strafe_weight	0.0 - 1.0	Peso del movimento laterale
8	retreat_health	20 - 70	Soglia salute per ritirata tattica
9	predictive_aim	0.0 - 1.0	Uso della mira predittiva
10	burst_fire	0.0 - 1.0	Modalità burst (3 colpi rapidi) vs normale
11	corner_avoid	0.0 - 1.0	Tendenza ad evitare gli angoli

Funzione di Fitness

La fitness è una funzione multi-obiettivo che valuta le prestazioni complessive del bot:

Formula Completa

Fitness = Sopravvivenza + Offensiva + Efficienza + Bonus

Dove:
Sopravvivenza = health * 5 + time_alive * 2
Offensiva = damage_dealt * 30 + hits * 70
Efficienza = (hits / shots_fired) * 300
Bonus = 500 (se vivo alla fine)
I valori sono stati scelti affinchè i combattimenti risultino più avvincenti. Con maggior premio per la sopravvivenza e minor peso all' offensiva, i bot si evitavano tra di loro per sopravvivere entrambi.

Componenti della Fitness

- **Sopravvivenza (peso alto)** - health * 5: premia la salute residua - time_alive * 2: premia la longevità nel combattimento;
- **Capacità Offensiva** - damage_dealt * 30: danno totale inflitto - hits * 70: numero di colpi andati a segno;
- **Efficienza** - accuracy * 300: rapporto hits/shots (precisione) - Penalizza lo spreco di munizioni;
- **Bonus Vittoria** - +500 se il bot è vivo alla fine della battaglia - Incentiva fortemente la sopravvivenza.

Bilanciamento

Il sistema di fitness è progettato per evitare strategie estreme:

- Un bot puramente difensivo avrà bassa offensiva;
- Un bot puramente offensivo rischia di morire presto.

L'ottimo è un bilanciamento tra attacco e difesa.

Operatori Genetici

La Selezione: Il Metodo del Torneo

Per scegliere i genitori della nuova generazione, è stato utilizzato il metodo del **Tournament Selection** con una dimensione del torneo pari a 3.

toolbox.register("select", tools.selTournament, tournsize=3)

Come funziona in pratica:

1. **Campionamento:** Il sistema sceglie casualmente 3 robot dalla popolazione attuale.
2. **Duello di Fitness:** Tra questi 3, viene analizzato il punteggio di fitness e viene scelto il robot con il valore più alto.
3. **Ripetizione:** Questo processo viene ripetuto finché non abbiamo abbastanza vincitori per riempire la nuova popolazione.

È scelto questo metodo (Vantaggi):

- **Pressione selettiva moderata:** Scegliendo solo tra 3 individui, diamo una possibilità di riprodursi anche a robot che non sono i primi in classifica, ma che potrebbero avere dei geni interessanti.
- **Mantenimento della diversità:** Evita che il robot più forte in assoluto "monopolizzi" subito tutta la popolazione, rischiando di creare un esercito di robot tutti identici.
- **Prevenzione della convergenza prematura:** Impedisce al sistema di bloccarsi troppo presto su una strategia discreta, spingendolo a esplorare tattiche di combattimento sempre più avanzate.

Riproduzione e Variabilità
Dopo la selezione, i robot scelti generano la nuova popolazione attraverso due meccanismi biologici: il **Crossover** e la **Mutazione**.

1. Crossover (Incrocio a due punti)
Si è scelto di utilizzare la funzione `cxTwoPoint` della libreria **DEAP**. Questo metodo simula lo scambio di materiale genetico tra due individui per creare una prole che erediti le migliori caratteristiche di entrambi.
Esso sceglie casualmente due punti all'interno del genoma (la lista di 12 geni). Tutto ciò che si trova tra questi due punti viene scambiato tra i due genitori.
Esempio:

Genitore A: `[0.2, 0.5, |0.8, 0.3, 0.6|, 0.1]`
Genitore B: `[0.7, 0.1, |0.4, 0.9, 0.2|, 0.5]`
Scambio del segmento centrale...
Figlio A: `[0.2, 0.5, |0.4, 0.9, 0.2|, 0.1]`
Figlio B: `[0.7, 0.1, |0.8, 0.3, 0.6|, 0.5]`

2. Mutazione Gaussiana

La mutazione è fondamentale per evitare che i robot diventino tutti uguali e per permettere loro di "scoprire" tattiche mai viste prima.
Esso non cambia completamente un gene, ma gli aggiungiamo un piccolo "disturbo" casuale basato sulla **distribuzione Gaussiana** (una curva a campana).

- **Parametri:**
  - **$\mu = 0$ (Media):** La mutazione è bilanciata; il valore del gene può aumentare o diminuire con la stessa probabilità.
  - **$\sigma = 0.2$ (Deviazione standard):** Determina quanto è forte il cambiamento. Un valore di 0.2 significa che la maggior parte delle mutazioni sposterà il gene del $\pm 20\%$.
  - **indpb = 0.2:** Ogni singolo gene all'interno di un robot ha il 20% di probabilità di subire questa modifica.

Effetto pratico:

Questo metodo introduce una **variabilità continua**. I robot non cambiano radicalmente strategia da un momento all'altro (il che distruggerebbe ciò che hanno imparato), ma "aggiustano il tiro" gradualmente, perfezionando i propri riflessi e parametri di combattimento.

Sistema di Simulazione

L'arena di gioco e le meccaniche di combattimento sono state progettate per testare l'efficacia delle strategie evolute dai bot in un ambiente controllato e imparziale.

L'Arena e le Regole di Ingaggio

L'ambiente di combattimento è uno spazio rettangolare di **800×600 pixel**.

- **Durata Match:** Ogni scontro è limitato a **600 frame** (circa 10 secondi). Questo evita loop infiniti e forza i bot a essere aggressivi.
- **Sistema di Salute (HP):** I bot iniziano con **100 HP**. Un proiettile a segno sottrae **10 HP**.
- **Fisica dei Bordi:** Se un bot tocca i confini dell'arena, rimbalza, impedendo ai robot di "incastrarsi" negli angoli.

Bilanciamento delle Battaglie

Per eliminare qualsiasi pregiudizio statistico dovuto alla posizione di partenza, è stato implementato un sistema di **battaglie speculari**:

1. **Round A:** Il Team Rosso parte a sinistra, il Blu a destra.
2. **Round B:** Le posizioni vengono invertite.
3. **Fitness:** Il punteggio finale è la **media dei due scontri**. Questo assicura che un bot sia considerato "forte" solo se vince indipendentemente dalla sua posizione iniziale.

Logica di Aggiornamento del Bot

Ad ogni frame, il robot segue un ciclo di sei fasi:

- **Percezione:** Calcola matematicamente dove si trova il nemico.
- **Decisione:** Il robot sceglie se caricare, scappare o muoversi lateralmente (movimento circolare). Se vede un proiettile nemico, attiva la logica di **evasione**.

• **Movimento & Mira:** Si sposta e punta l'arma. La precisione della mira dipende dai suoi geni.
• **Sparo:** Valuta se il nemico è a tiro e se il sistema di raffreddamento (*cooldown*) permette di fare fuoco.

Fisica dei Proiettili e Collisioni

I proiettili seguono una fisica balistica lineare semplice:$$bullet.x += \cos(angle) \times 8$$$$bullet.y += \sin(angle) \times 8$$

La gestione delle collisioni è di tipo **circolare**: se la distanza tra il centro del proiettile e il centro del robot è inferiore alla somma dei loro raggi, il colpo è considerato a segno.

Risultati e Osservazioni

Analisi dei Risultati Evolutivi

L'evoluzione dei Battle Bots ha mostrato una progressione chiara, passando da movimenti puramente casuali a strategie tattiche complesse.

1. Fasi dell'Evoluzione

Il processo evolutivo può essere suddiviso in quattro momenti chiave:

• **Fase Caotica (Gen. 1-5):** I bot si muovono senza una meta e sparano casualmente. La fitness è bassa ($500-800$) poiché molti match finiscono in pareggio o con pochissimi danni inflitti.
• **Emergenza di Pattern (Gen. 10-20):** Inizia la selezione naturale. I bot che mantengono la distanza sopravvivono di più. La precisione aumenta e la fitness raddoppia ($1200-1800$).
• **Strategie Sofisticate (Gen. 30-50):** Compaiono comportamenti avanzati come il **movimento circolare** e la **mira predittiva** (sparare dove il nemico sarà, non dove si trova).
• **Ottimizzazione (Gen. 50+):** La fitness si stabilizza ($2500-3500$). I bot hanno ormai raggiunto un "plateau" di competenza dove i miglioramenti sono minimi ma costanti.

2. Archetipi Evolutivi (I "Personaggi")

L'algoritmo non ha prodotto un unico tipo di robot, ma diverse "classi" di combattenti:

Archetipo	Caratteristica Chiave	Strategia
Il Cecchino	Mira > 0.8	Colpisce da lontano e resta ai margini dell'arena.
Il Tank	Velocità > 4.5	Carica l'avversario e spara raffiche a bruciapelo.
Il Danzatore	Evasione > 0.7	Difficilissimo da colpire grazie a movimenti a spirale.
Il Tattico	Bilanciato	Cambia stile e batte in ritirata se la salute scende sotto il 40%.

3. Risoluzione dello Sbilanciamento (Bias)

Inizialmente è stato riscontrato un forte squilibrio: il **Team Rosso vinceva il 73% delle volte**.

• **Problema:** Essendo il Rosso sempre a sinistra, beneficiava di un vantaggio posizionale costante o di un bias nel calcolo delle coordinate.
• **Soluzione:** Implementazione del **Sistema di Battaglie Speculari**. Facendo combattere ogni coppia due volte con posizioni invertite, il tasso di vittoria si è stabilizzato intorno al **50%**, garantendo che l'evoluzione fosse basata sul merito genetico e non sulla posizione.

4. Convergenza e Diversità

L'equilibrio del sistema è garantito dal trade-off tra:

• **Convergenza:** La stabilità della fitness dopo la 60ª generazione indica che i bot hanno trovato strategie "ottime".

• **Diversità:** Grazie alla mutazione al 30% e al torneo non troppo restrittivo, la popolazione non diventa mai piatta; c'è sempre un nuovo "mutante" che prova a sfidare i campioni in carica.

Conclusioni e Sviluppi Futuri

Conclusioni e Prospettive Future

Il progetto **Battle Bots Arena** ha confermato con successo che l'applicazione degli algoritmi genetici permette l'emersione di comportamenti complessi e adattivi partendo da una base di regole semplici e parametri casuali.

Analisi Critica

L'utilizzo della libreria **DEAP** accoppiata a un motore grafico in **Pygame** ha permesso di creare un framework robusto per osservare la selezione naturale in tempo reale.

- **Punti di Forza:** Il sistema di fitness bilanciato ha eliminato i vantaggi ambientali, permettendo un'evoluzione basata esclusivamente sul merito strategico. La varietà di archetipi osservati (Sniper, Tank, Dancer) dimostra la capacità del sistema di esplorare diverse nicchie evolutive.
- **Limitazioni:** Attualmente, la "memoria" dei bot è nulla: ogni decisione è basata solo sul frame corrente. Inoltre, la fisica semplificata e lo spazio di ricerca limitato a 12 geni restringono la complessità massima raggiungibile dalle strategie.

Sviluppi Futuri

Per superare gli attuali limiti e aumentare la profondità del simulatore, sono state identificate diverse direzioni di sviluppo:

1. **Ampliamento del Gameplay:** Introduzione di abilità attive (scudi temporanei, scatti direzionali) e mine, che richiederebbero geni aggiuntivi per la gestione del *timing*.
2. **Complessità Ambientale:** Inserimento di ostacoli statici (muri) e dinamici (power-up), trasformando l'arena da uno spazio vuoto a un ambiente tattico.
3. **Dinamiche di Squadra:** Evoluzione di battaglie **2v2 o 3v3**, dove la fitness non premierebbe più solo il singolo, ma la capacità di coordinamento e cooperazione.
4. **Apprendimento Online:** Integrare reti neurali (Neuroevoluzione) per permettere ai bot di adattarsi ai movimenti dell'avversario *durante* la singola battaglia, e non solo attraverso le generazioni.